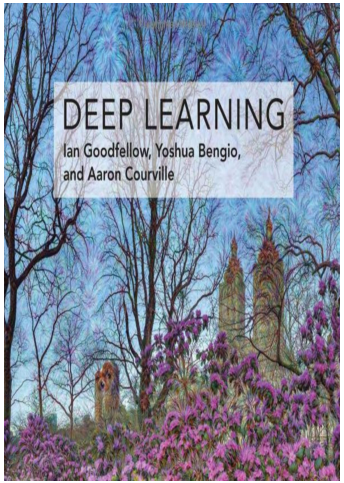
Ciencia de Datos
Igeniería Industrial
UTN FRBA
curso I5521

Clase_10: Redes Neuronales

UTN.BA INGENIERÍA INDUSTRIAL

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

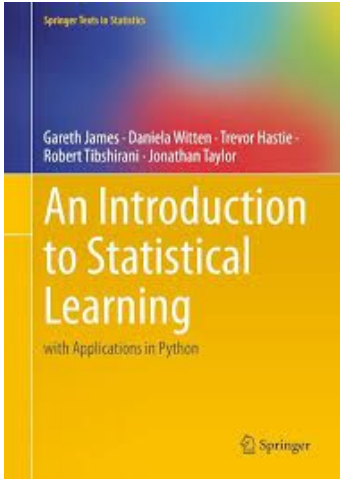
Lectura sugerida



Deep Learning
https://www.deeplearningbook.org/
Uno de los libros más importantes en redes neuronales contemporáneas

- Part II: Modern Practical Deep Networks
  - Capitulo 6: Deep Feedforward Networks
  - Capitulo 8: Optimization for Training Deep Models
  - Capitulo 10: Sequence Modeling: Recurrent and Recursive Nets
  - Capitulo 11: Practical Methodology

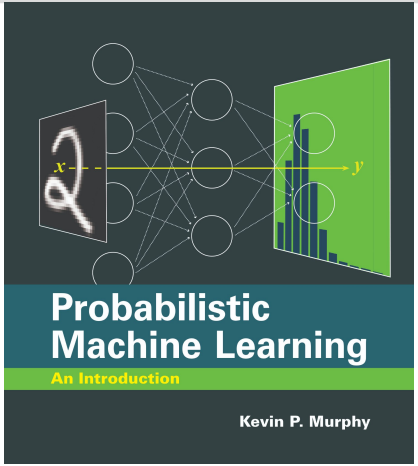
Lectura sugerida



Introduction to Statistical Learning (with applications in Python)
https://www.statlearning.com/
https://hastie.su.domains/ISLP/ISLP_website.pdf.download.html

- Capítulo 10 -> Deep Learning

Lectura sugerida



Probabilistic Machine Learning: An Introduction
https://probml.github.io/pml-book/book1.html

- Capítulo 13: Neural Networks for Tabular Data

Agenda

Redes Neuronales

- Perceptrón
- Funciones de activación
- Multilayer Perceptrón (MLP)
- Arquitecturas de Redes Neuronales (NN)
- Entrenamiento de una NN

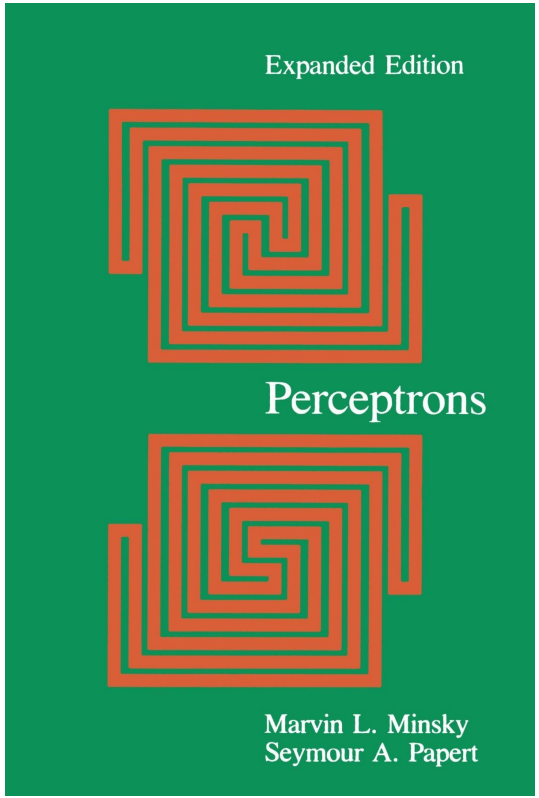
¿Que es una red neuronal artificial?

$$y = f_w(x)$$

$$w = \underset{w}{\operatorname{argmin}}\ L(y, \hat{y}) = \underset{w}{\operatorname{argmin}}\ L(y, \hat{f}_w(x))$$

Una red neuronal, es una función **f_w(x)** lineal o no lineal caracterizada por un conjunto de parámetros “W”. Esta función toma como entrada un vector **x** d-dimensional para generar una respuesta/salida unidimensional ‘y’ o multidimensional ‘z’. Los parámetros **w** se ajustan resolviendo un problema de optimización tal que minimice una función de costo **L** entre la respuesta ‘y’ del dataset y la respuesta de la red y_hat.

Perceptrón



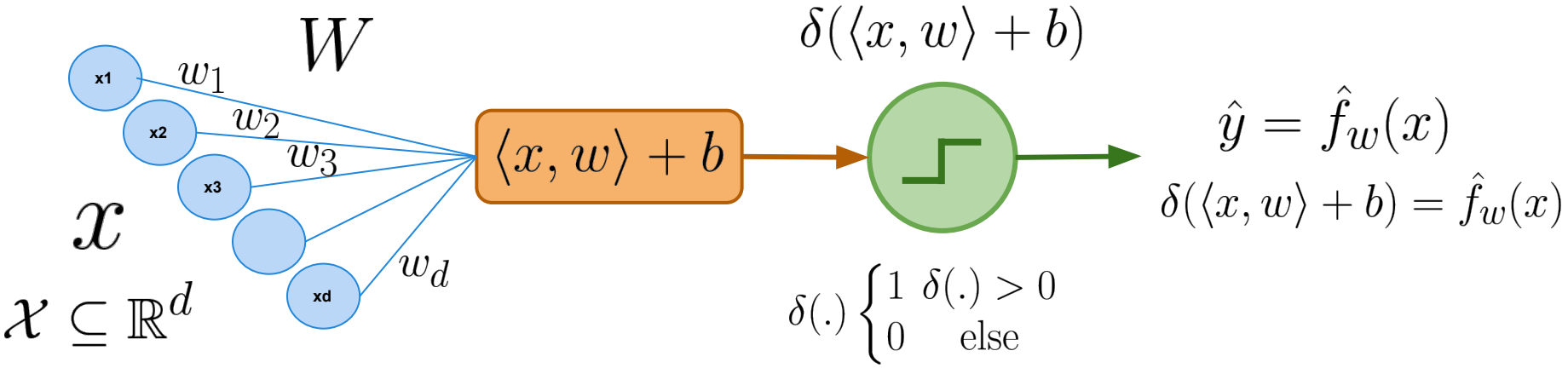
Psychological Review
Vol. 65, No. 6, 1958

THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN[1]

F. ROSENBLATT

Cornell Aeronautical Laboratory

Perceptrón



Para entender las redes neuronales primero debemos estudiar el modelo **perceptrón.** Es un algoritmo de Clasificación Lineal originado en los años 50s. El modelo tiene **d** features/variable de entradas, donde cada feature de entrada estará combinada linealmente a un "peso **w**". Es decir que tenemos una combinación lineal **(x . w)**. Una vez realizada la combinación lineal se determina el valor de salida **(w*x+b) = y.** Al resultado de la combinación lineal se lo afecta por una función de activación "step" que en el caso de ser binario resultará como 0 o 1.

El objetivo es aprender los pesos **w,b** que minimizan el error de clasificación **y_hat.**

Perceptrón

- Es un modelo que trabaja “online”. Procesa una muestra a la vez. De esta manera los pesos también se actualizan gradualmente.
- Únicamente en los casos donde el modelo realiza una predicción errónea los pesos se actualizan.
- El perceptrón computa una función de costo **L(y, y_hat)** basada en la cantidad de veces que hubo una mal clasificación.
- La función de activación (‘neurona’) que utiliza es “signo o step” -> es 0 si x<0 y +1 si x>0.

Neurona

Las NN artificiales, inspiradas en su contraparte biológica, el cerebro humano, son aproximadores de funciones y se clasifican como modelos paramétricos.

$$z(s) = \sigma\left(\sum_{j=1}^{n_{in}} W_j s_j + b\right) \tag{1}$$

La Eq. 1 proporciona la formulación básica de una sola neurona:
Donde z y s_j se refieren a la salida y la entrada de la neurona, y W_j y b se refieren a los parámetros que se pueden aprender. La agregación lineal de la entrada s_j ponderada por los parámetros W_j y b se denomina comúnmente *preactivación* de la neurona. Para enriquecer la complejidad de las funciones que la **NN** puede aproximar, la *preactivación* se utiliza como entrada de una función de activación $\sigma(\ )$. Las funciones de activación añaden capacidad no lineal a la **NN** que simula la estimulación producida por una neurona.

Ejercicio: Perceptron

$$\mathbf{x}_{test} = \begin{matrix} x_1 & x_2 \end{matrix} \begin{bmatrix} 1 & 1 \\ 3 & 0.5 \\ 2 & 3 \\ 2.5 & 1.5 \\ 1.5 & 2 \\ 2.5 & 4 \\ 3.5 & 1.5 \\ 4 & 3 \end{bmatrix} \quad \mathbf{y}_{test} = \begin{bmatrix} -1 \\ -1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

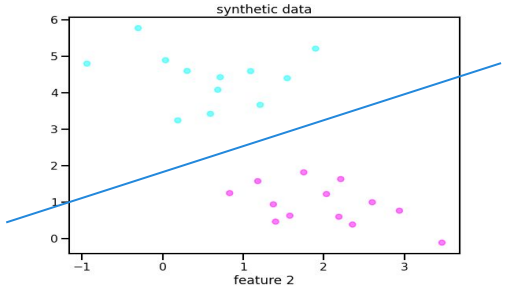
$$\delta(\langle x, w \rangle + b) = \hat{f}_w(x)$$

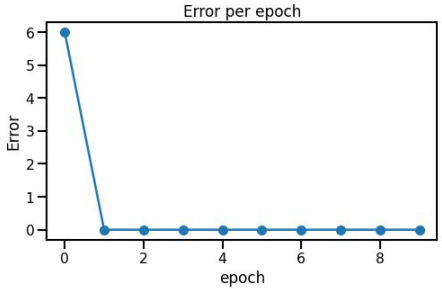
	w1	w2	b	Accuracy
$\hat{f}_1$	2	2	-9	?
$\hat{f}_2$	3.4	5	-14	?

Suponer dos modelos perceptrón f_1 y f_2 ya entrenados previamente con un dataset que no disponemos. Por lo tanto los parámetros de los modelos están disponibles. Se solicita evaluar los dos modelos utilizando las muestras de test provistas y calcular el accuracy para determinar cual de los dos generaliza mejor.

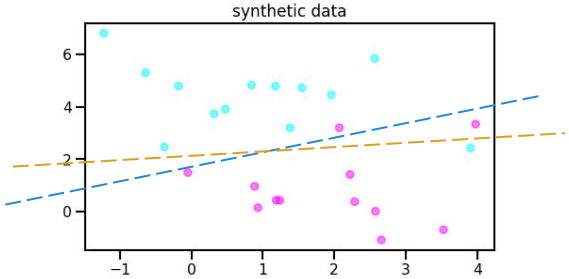
Limitaciones del perceptrón

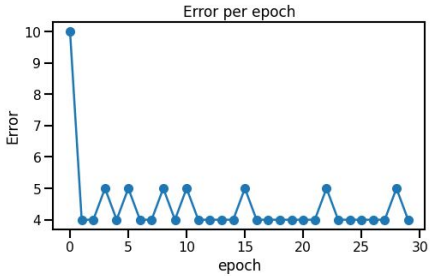
Linearly separable classes





Non Linear separable classes





Durante el entrenamiento el dataset es procesado por el perceptrón múltiples veces iterativamente donde cada iteración se la denomina “epoch”. El perceptrón únicamente actualiza los parámetros cuando el output **y_hat** es distinto a la verdadera etiqueta **y**. En clases linealmente separables el perceptron convergerá a una solución. En clases no separables el perceptrón iterara la cantidad de veces que el usuario le indique sin poder llegar a una solución definitiva.

Aprendizaje Supervisado



pic01 X_1

$f(x) = y$ → DOG y_1



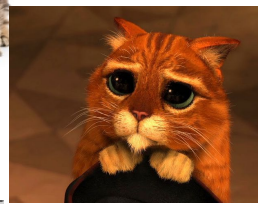
pic02 X_2

$f(x) = y$ → DOG y_2



pic03 X_3

$f(x) = y$ → CAT y_3



pic04 X_4

$f(x) = y$ → CAT y_4

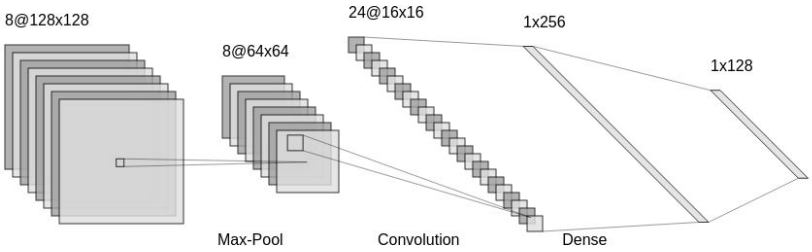
Data type: images
Features: pixels

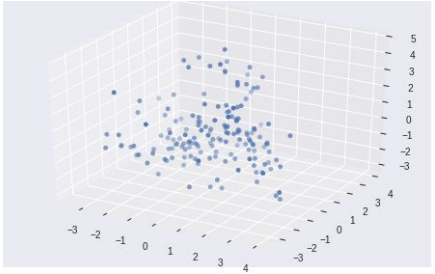
cada instancia esta asociada a una etiqueta determinada por un humano.

*Imagen: Palazzo

Aprendizaje Supervisado



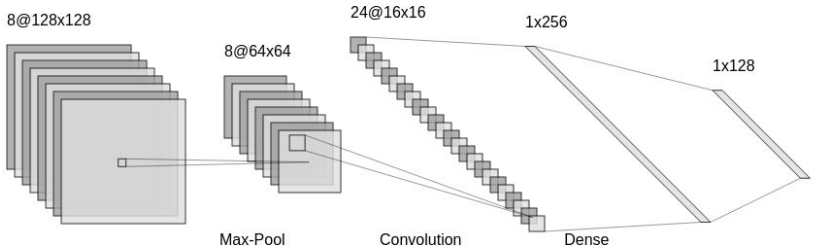




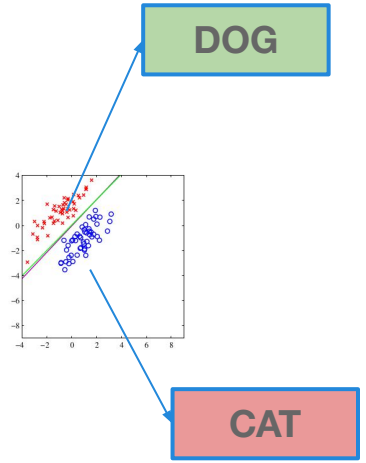
Aprendizaje Supervisado



$$f_w(x) = y$$



Red Neuronal convolucional



El modelo que aprendemos mapea las imágenes a un espacio de coordenadas sencillo de manera tal que las categorias de imagenes se separen y sean facil de clasificar.

*Imagen: Palazzo

Feature extraction (representation learning)



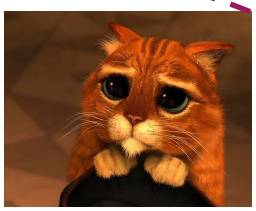
pic01 X_1



pic02 X_2

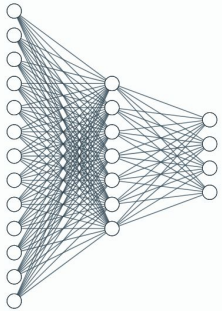


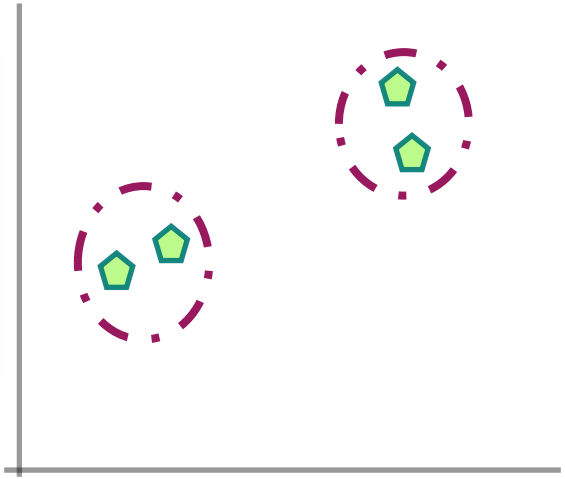
pic03 X_3



pic04 X_4

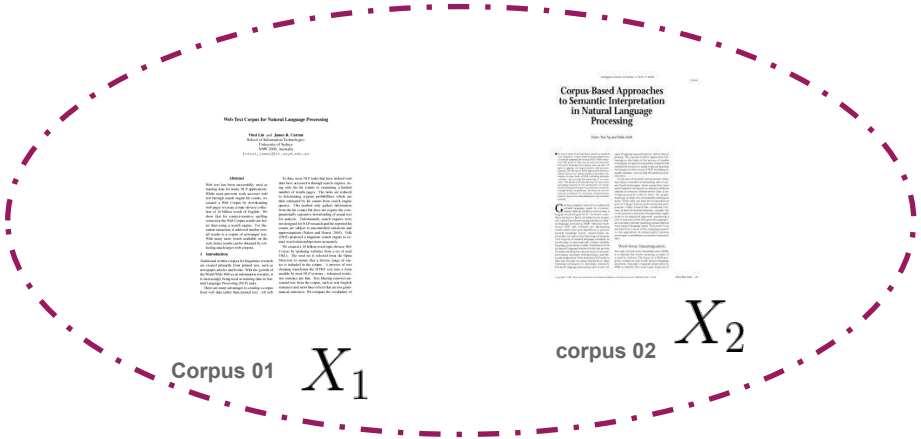
Pixel domain

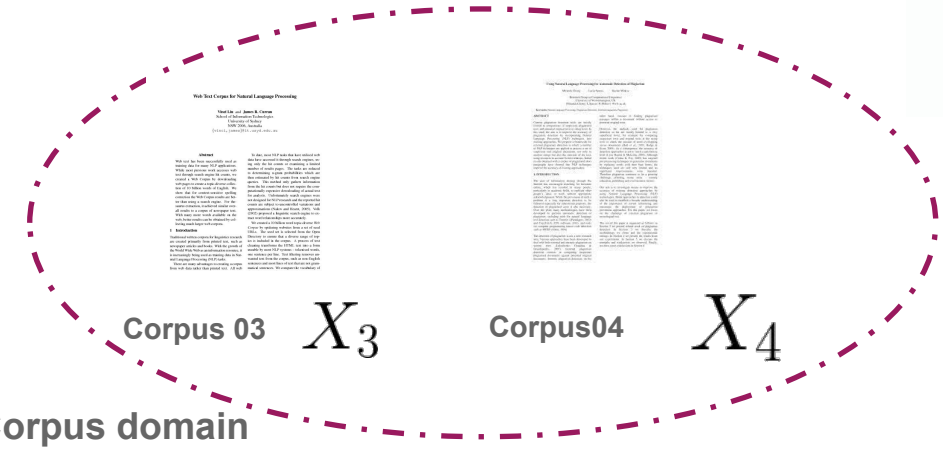




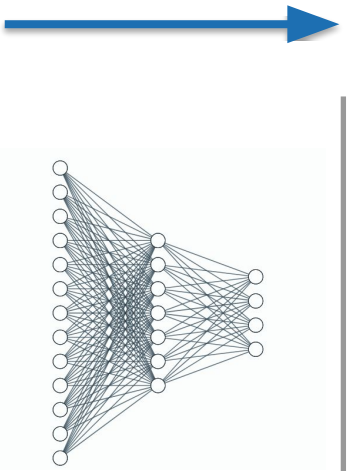
Low dimensional latent space domain

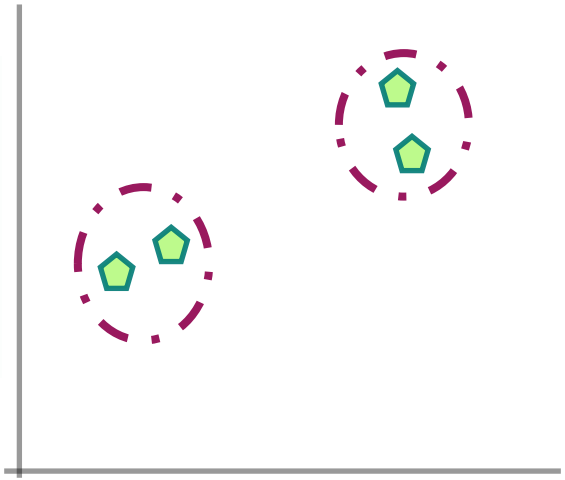
Feature extraction (representation learning)





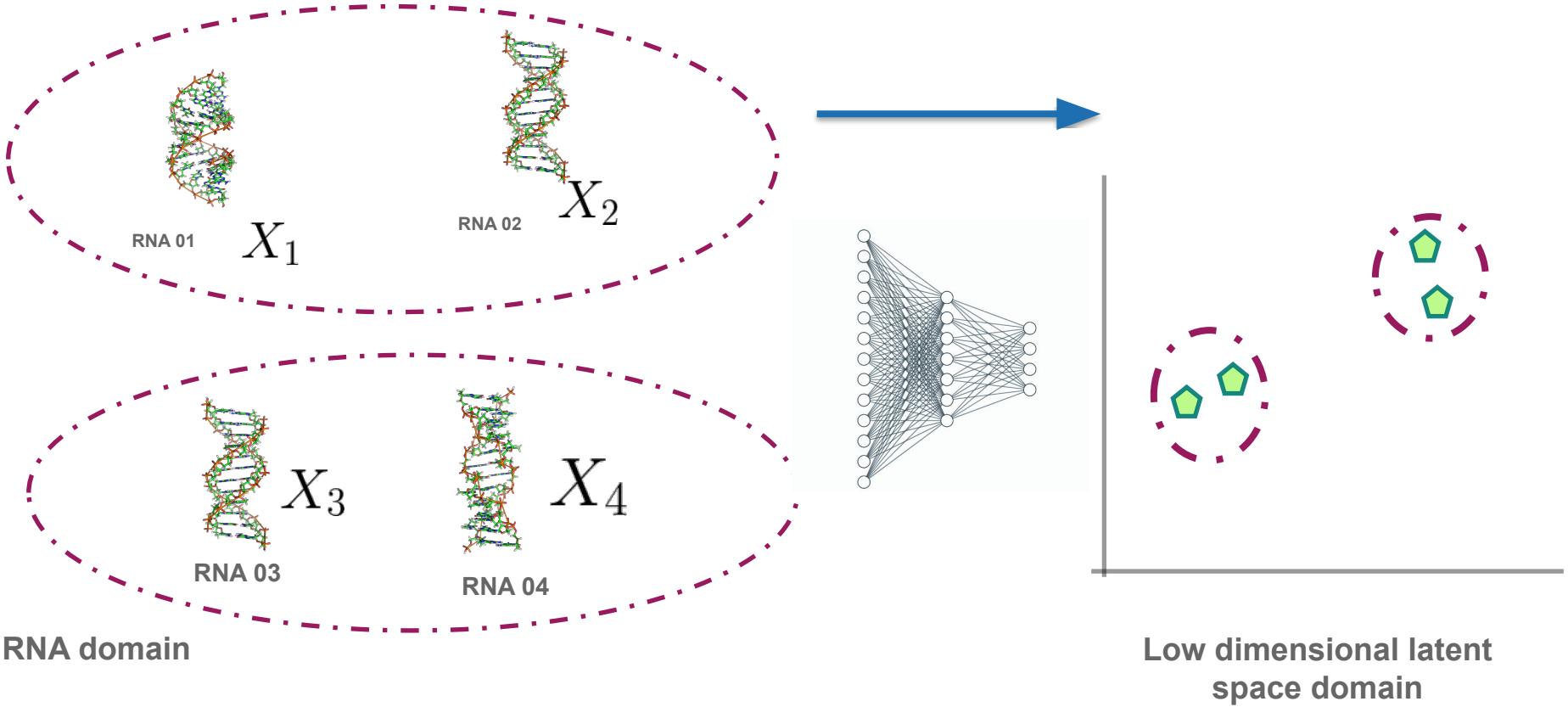
Corpus domain





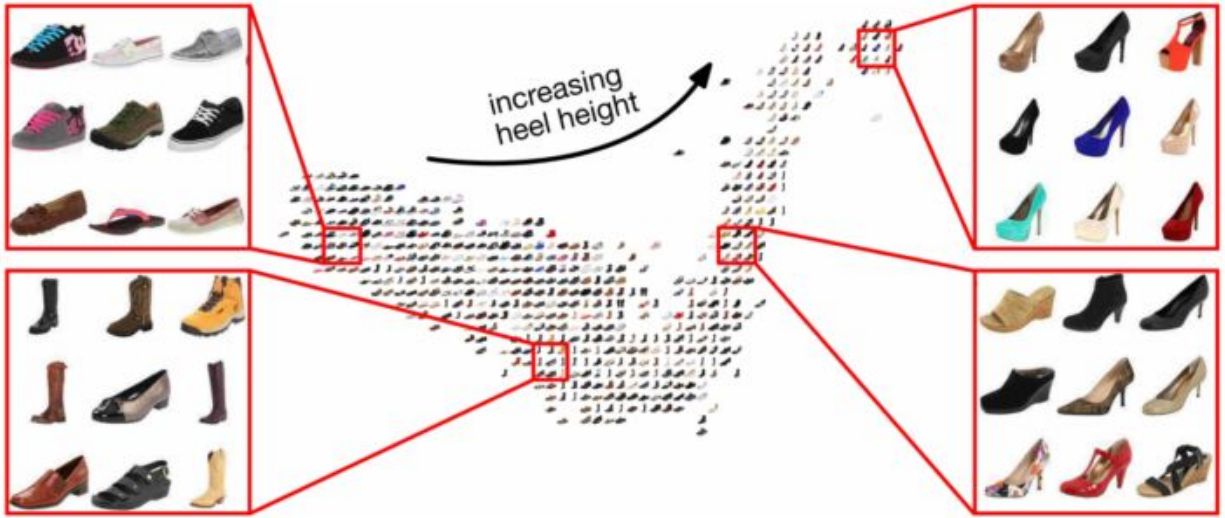
Low dimensional latent space domain

Feature extraction (representation learning)



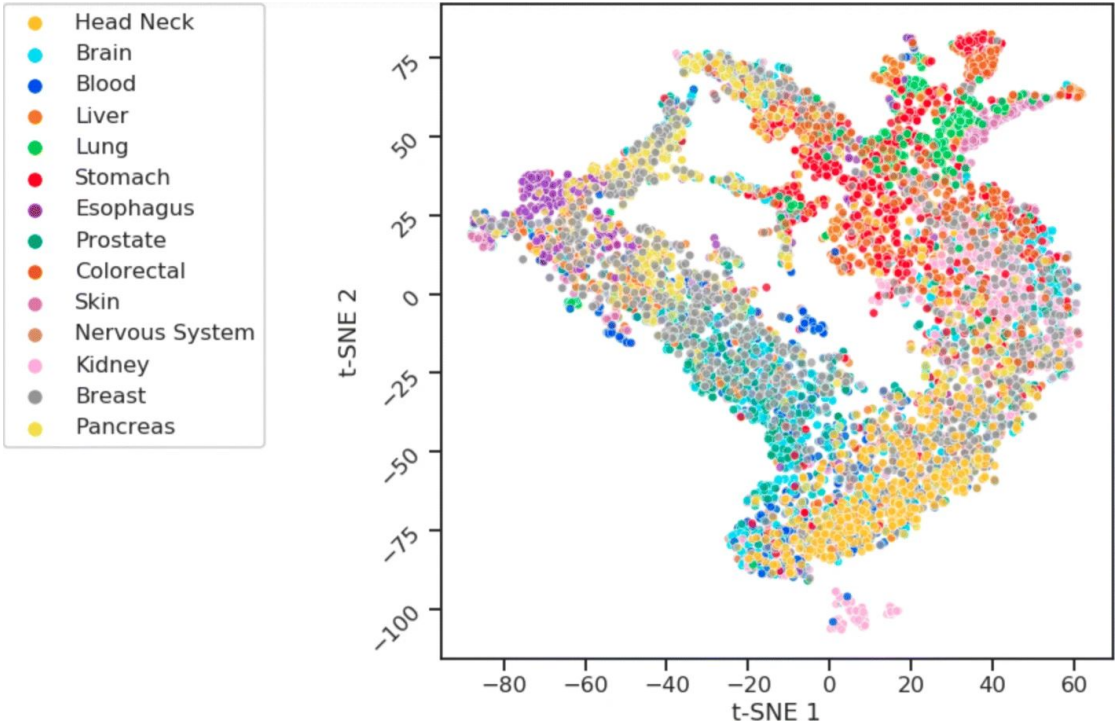
Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, *35*(8), 1798-1828.

Image embedding

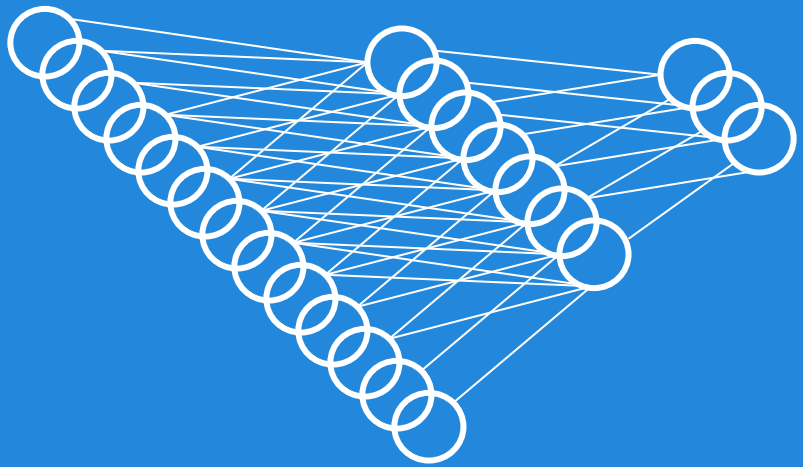


https://vision.cornell.edu/se3/embeddings-and-metric-learning/

Tumor embedding (tum2vec)

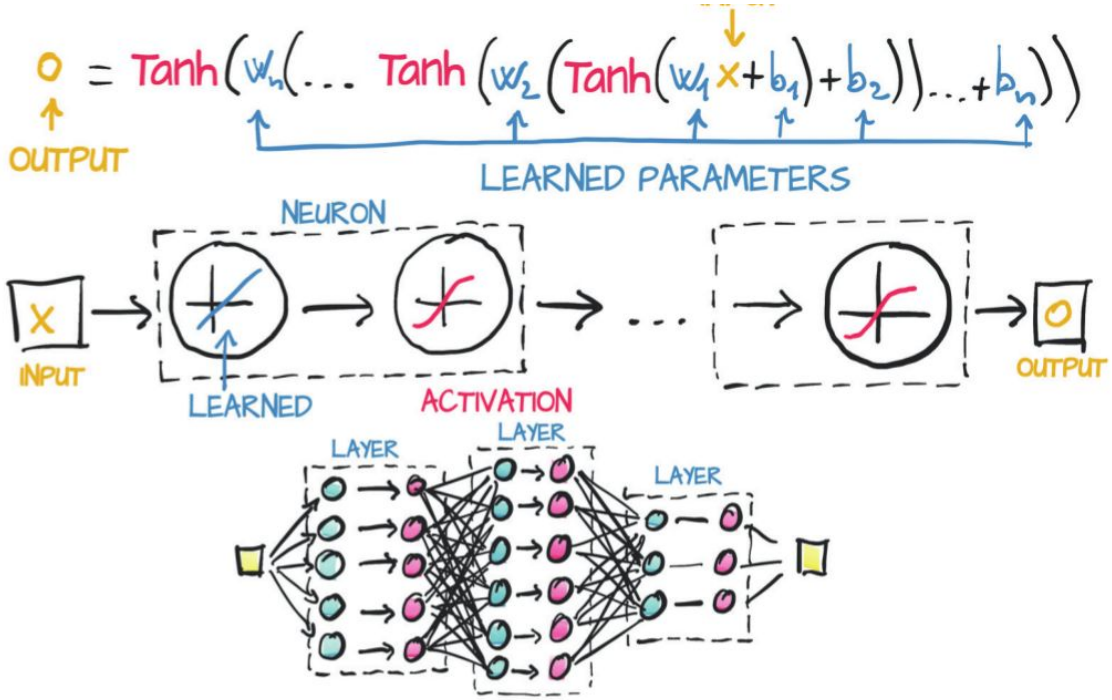


* Palazzo, et. Al.. A pan-cancer somatic mutation embedding using autoencoders. *BMC Bioinformatics* 20, 655 (2019). https://doi.org/10.1186/s12859-019-3298-z



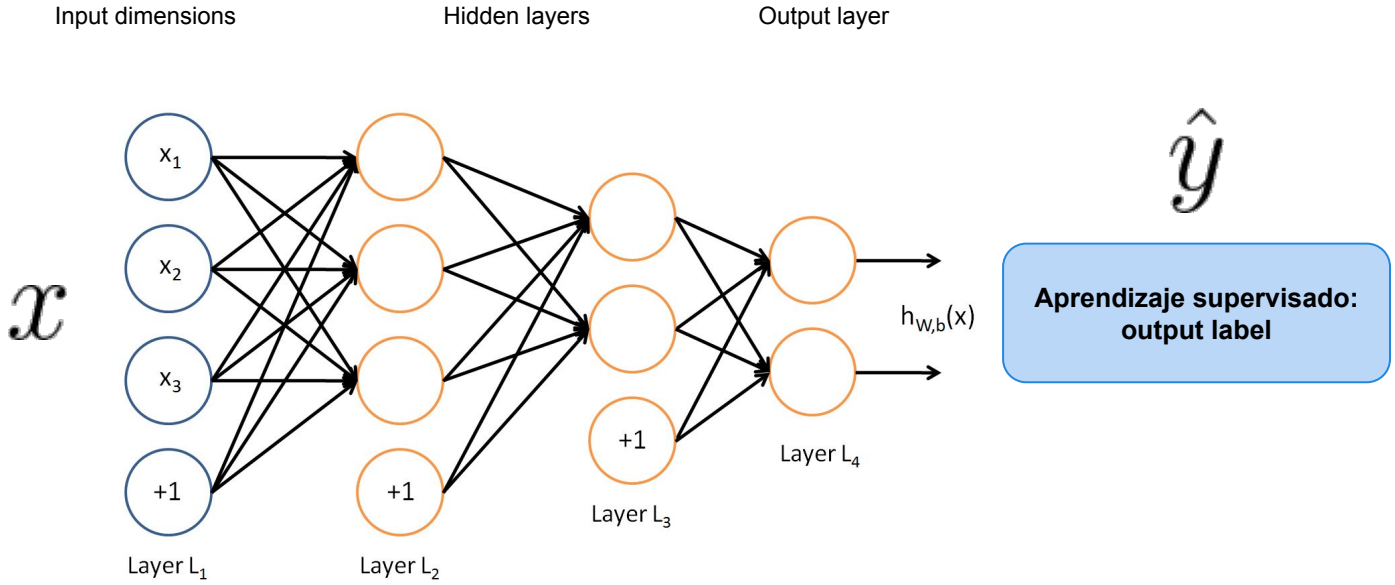
Redes Neuronales Artificiales

Neural Networks: Arquitectura



*Deep Learning with Pytorch, 2019

Neural Networks: Arquitectura



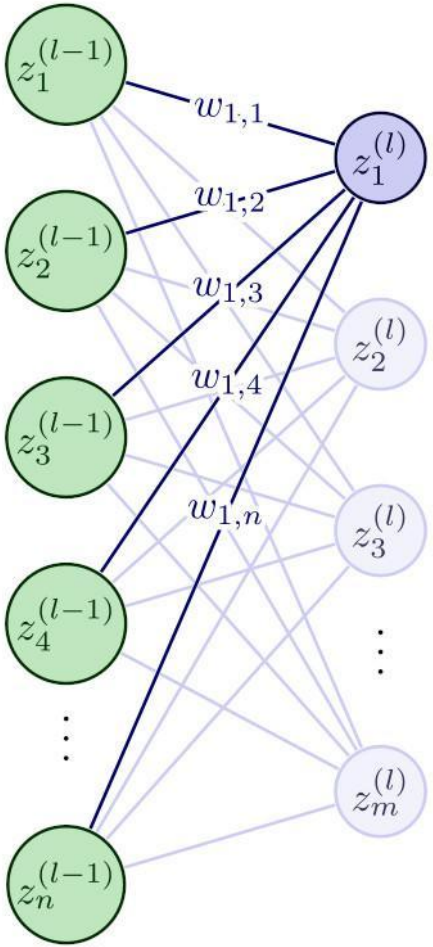
La arquitectura multi-capa o perceptrón multi-capa permite capturar patrones mas complejos en los datos. Cada capa es una representación de los datos donde cada neurona de una capa es una feature/variable aprendida por la combinación de variables de la capa anterior. A medida que mas capas se agregan mas complejas pueden ser las variables/features aprendidas. De todos modos agregar mas capas implica agregar mas complejidad y parámetros al modelo y de beneficio aprender representaciones mas beneficiosas de la distribución de los datos.

Multilayer Perceptrón / Feedforward Network

Cuando se combinan varias neuronas n_{out} que reciben el vector dimensional n_{in} s_j y producen el vector dimensional de activación, se crea una *capa*. Este punto de vista colectivo parametriza una capa mediante un vector de sesgos b_i y una matriz de pesos W_{ij}, donde $i = 1, .., n_{out}$ y $j = 1, .., n_{in}$, junto con una función de activación fija $\sigma(\ )$. El apilamiento de muchas capas define una de las arquitecturas básicas de las **NN**, la **MLP**. En la Ec. 2 se puede encontrar la formulación matemática de un **MLP** para la capa l.

$$z_i^{(l)} = \sigma\left(\sum_{j=1}^{n_{l-1}} W_{ij}^{(l)} z_j^{(l-1)} + b_i^{(l)}\right) \begin{cases} \forall i = 1, ..., n_l \\ \forall l = 1, ..., L \end{cases} \tag{2}$$

Multilayer Perceptrón / Feedforward Network



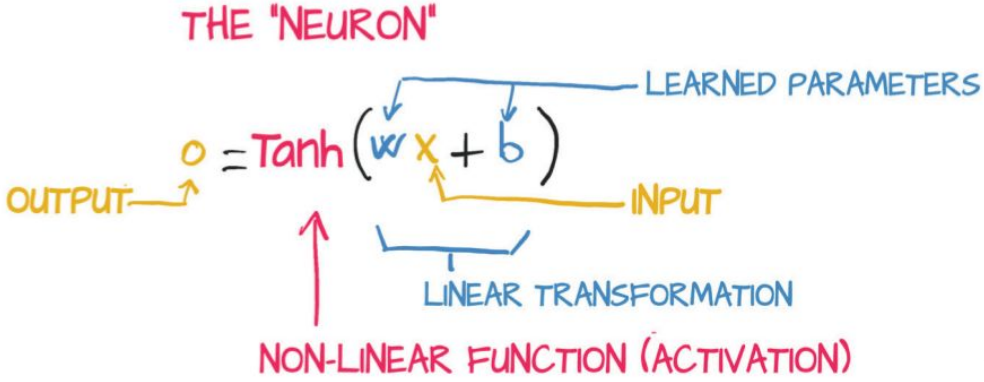
$$= \sigma\left(w_{1,1}z_1^{(l-1)} + w_{1,2}z_2^{(l-1)} + \ldots + w_{1,n}z_n^{(l-1)} + b_1^{(l)}\right)$$

$$= \sigma\left(\sum_{j=1}^{n} w_{1,j}z_j^{(l-1)} + b_1^{(l)}\right)$$

$$\begin{pmatrix} z_1^{(l)} \\ z_2^{(l)} \\ \vdots \\ z_m^{(l)} \end{pmatrix} = \sigma\left[\begin{pmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,n} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,1} & w_{m,2} & \cdots & w_{m,n} \end{pmatrix}\begin{pmatrix} z_1^{(l-1)} \\ z_2^{(l-1)} \\ \vdots \\ z_n^{(l-1)} \end{pmatrix} + \begin{pmatrix} b_1^{(l)} \\ b_2^{(l)} \\ \vdots \\ b_m^{(l)} \end{pmatrix}\right]$$

$$\mathbf{z}^{(l)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}\right)$$

Funciones de activación

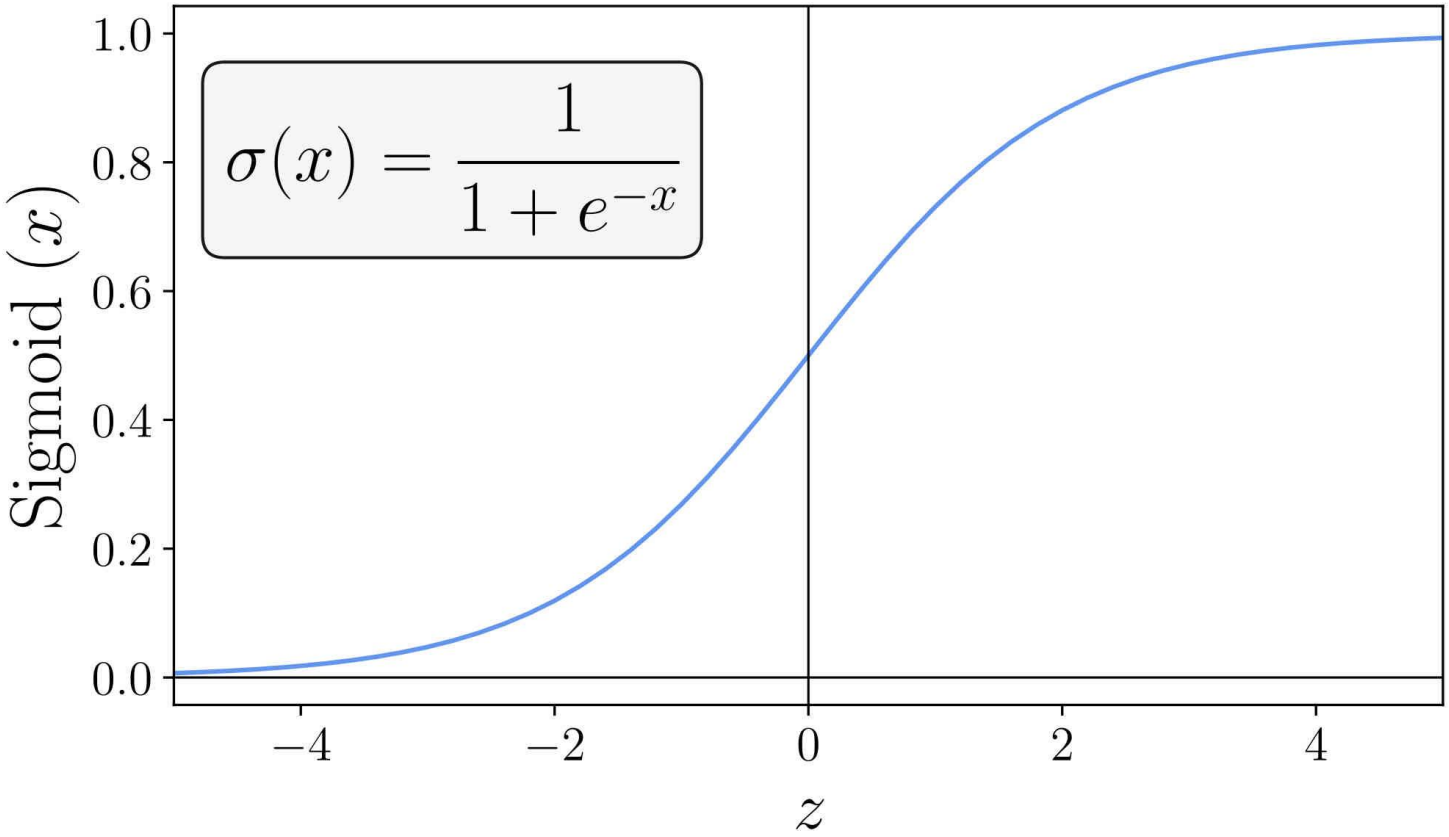


*Deep Learning with Pytorch, 2019

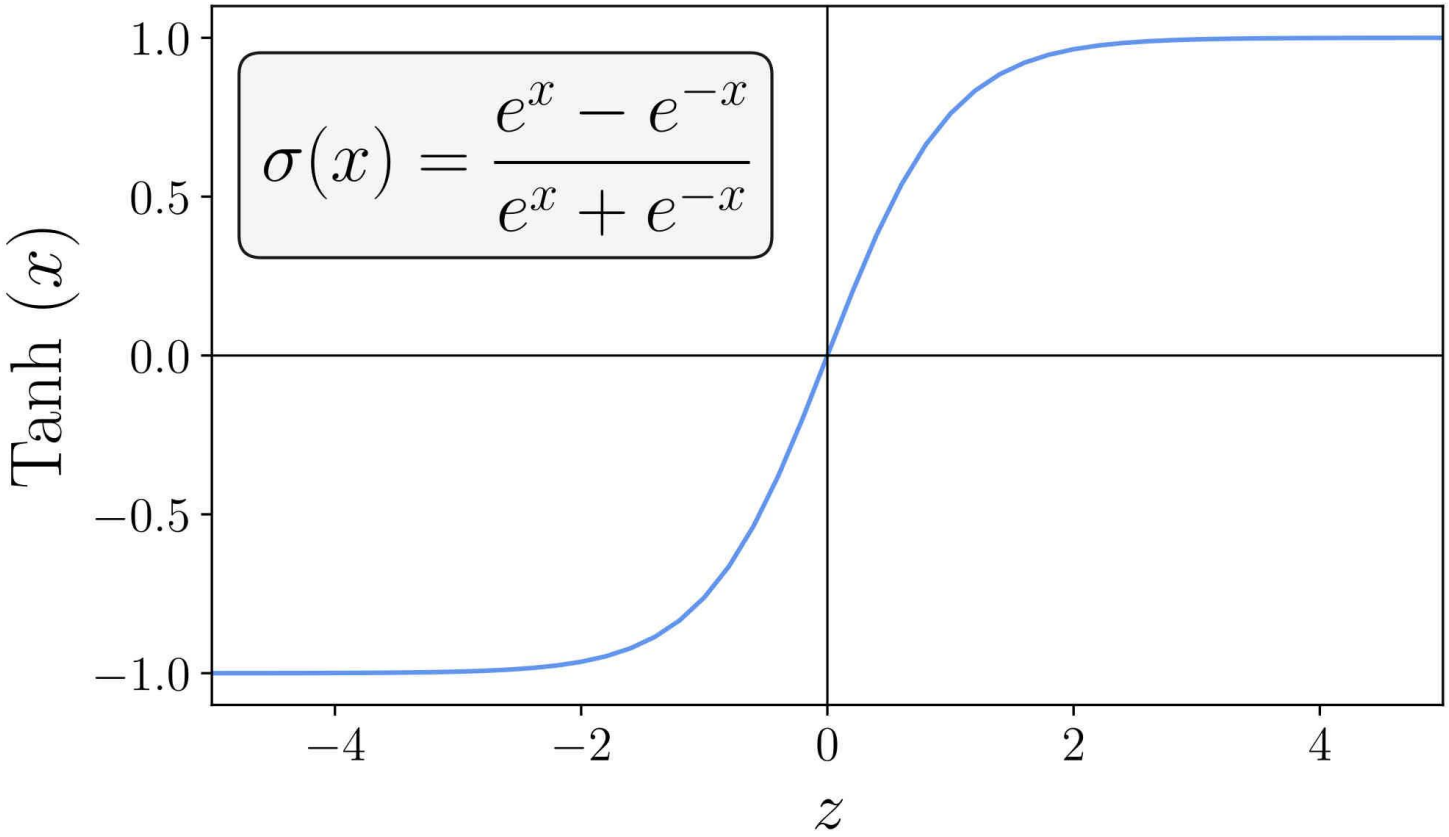
La función de activación es un **nodo o neurona** que tiene múltiples entradas y define una determinada salida. Las funciones de activación pueden estar “encendidas” o “desactivadas”. Algunas populares son:

- Sigmoid
- TanH
- ReLU
- Leaky-ReLu
- ELU
- Etc …

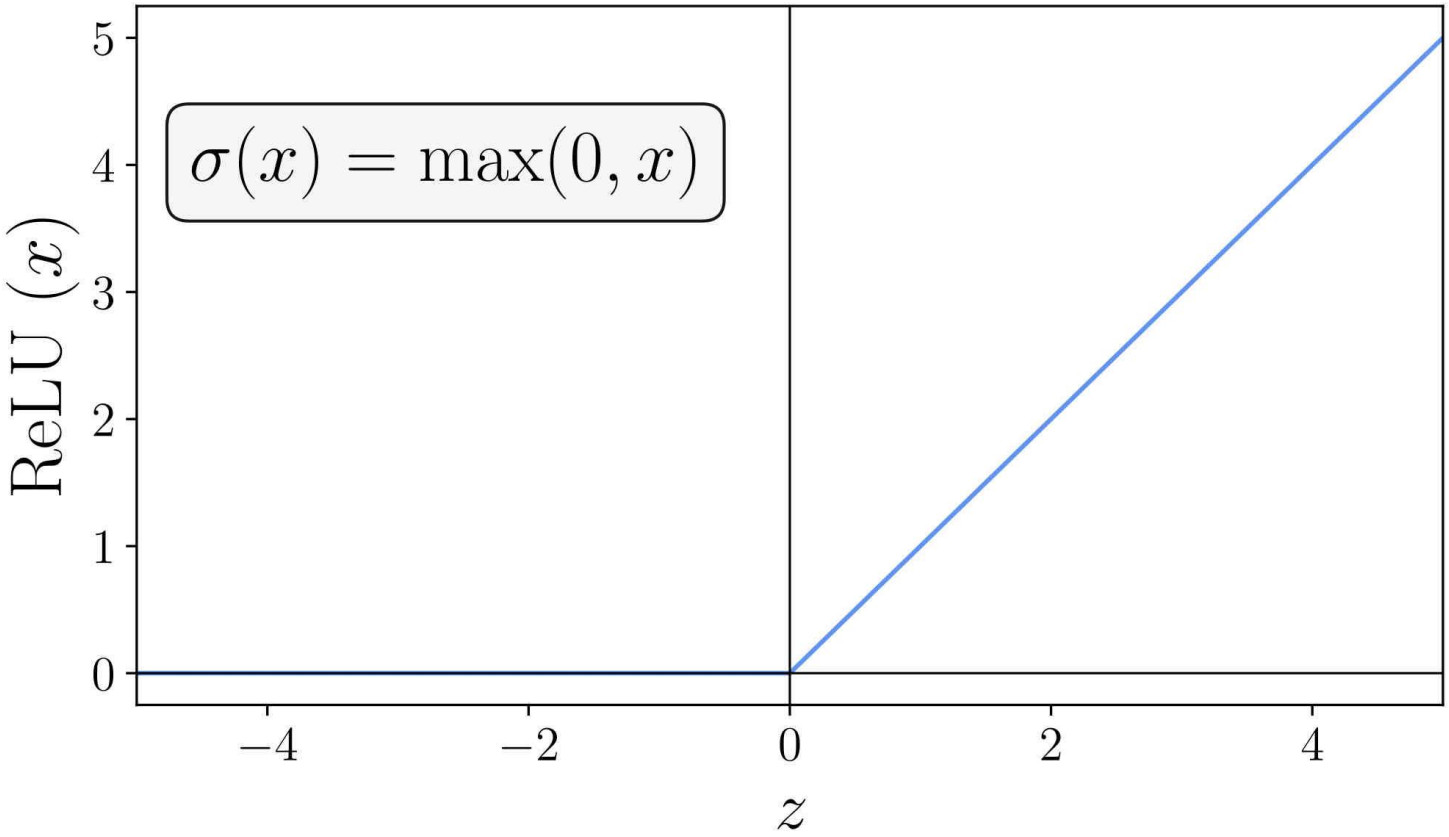
Funciones de activación



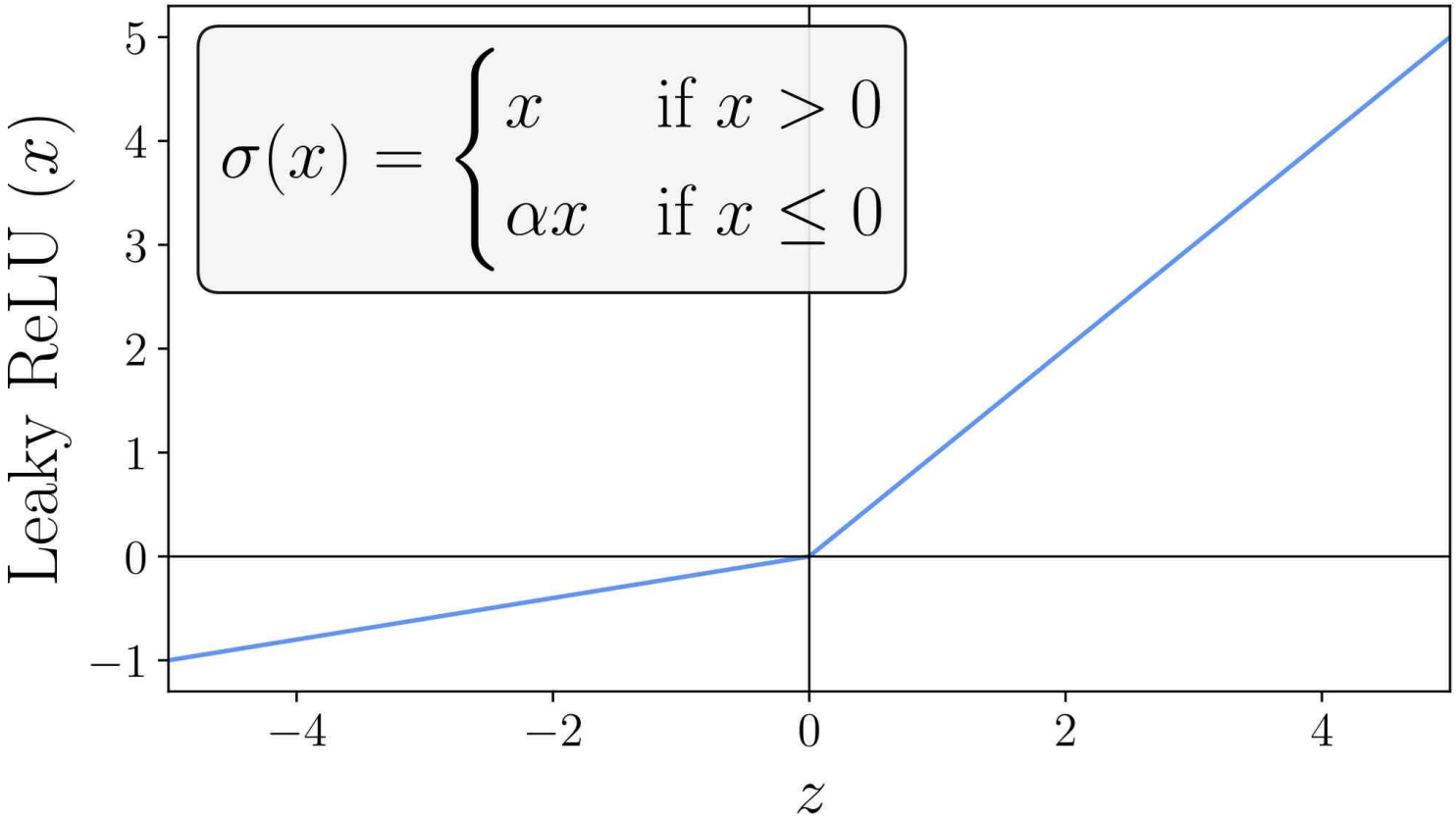
Funciones de activación



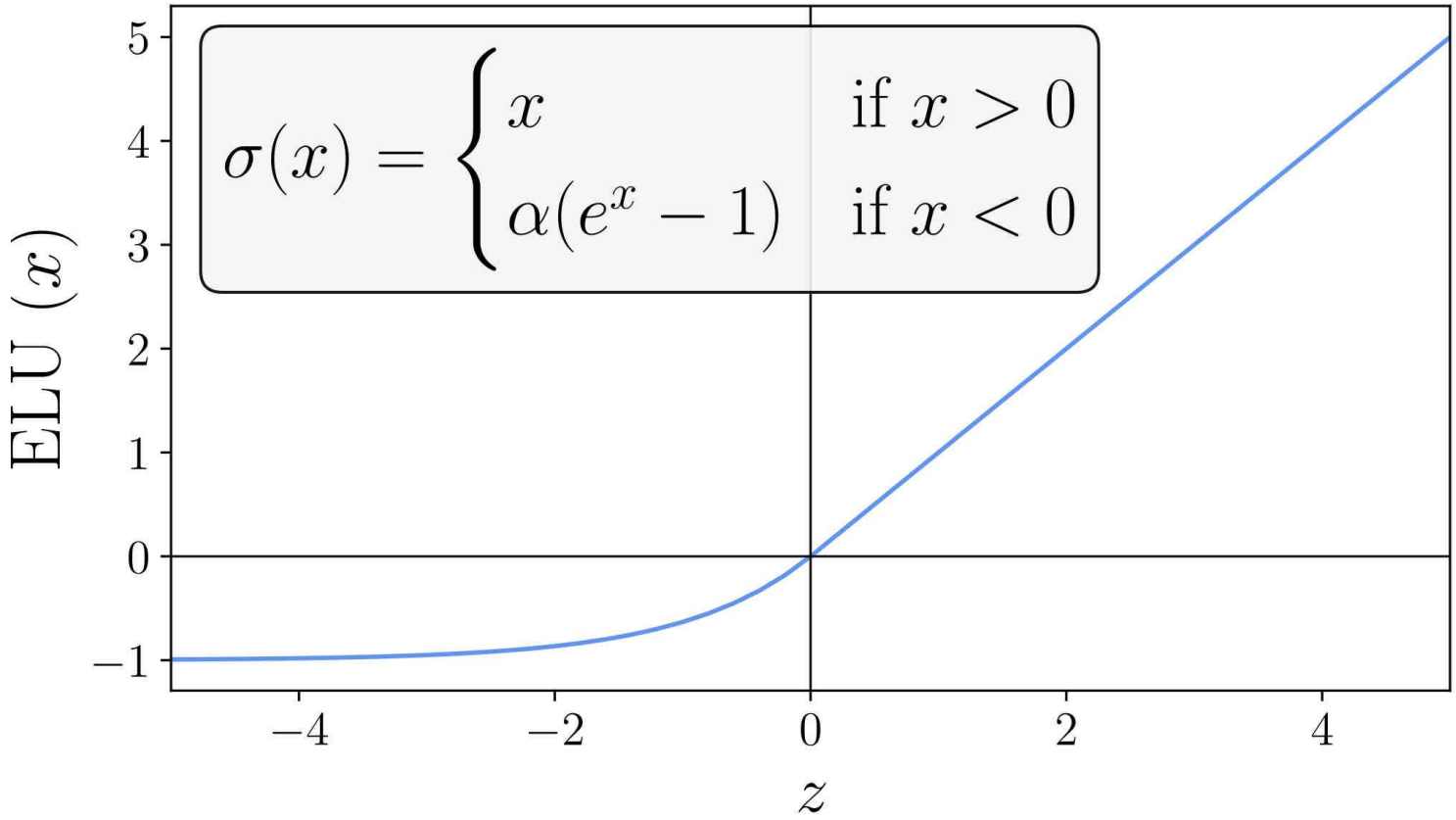
Funciones de activación



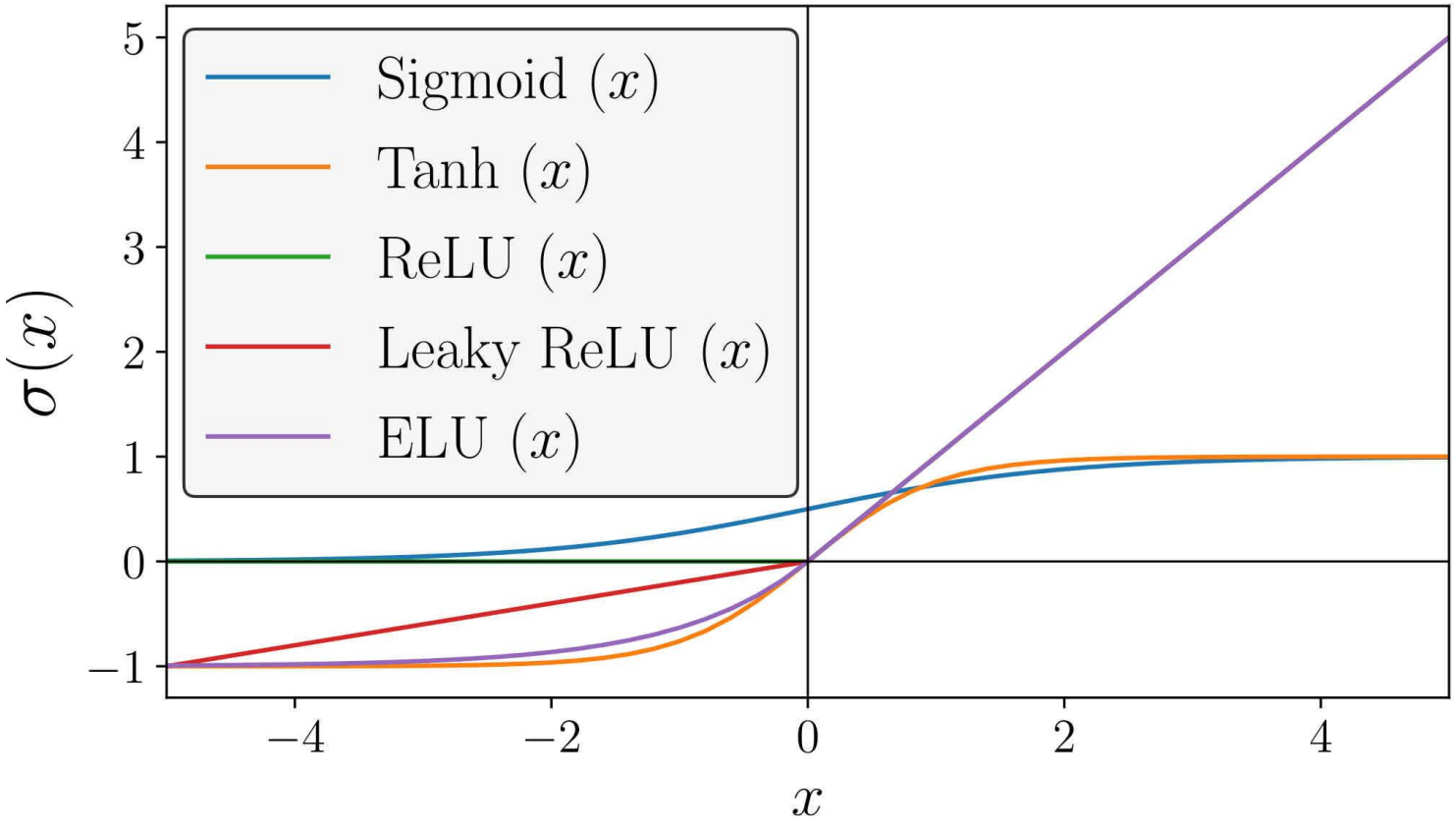
Funciones de activación



Funciones de activación

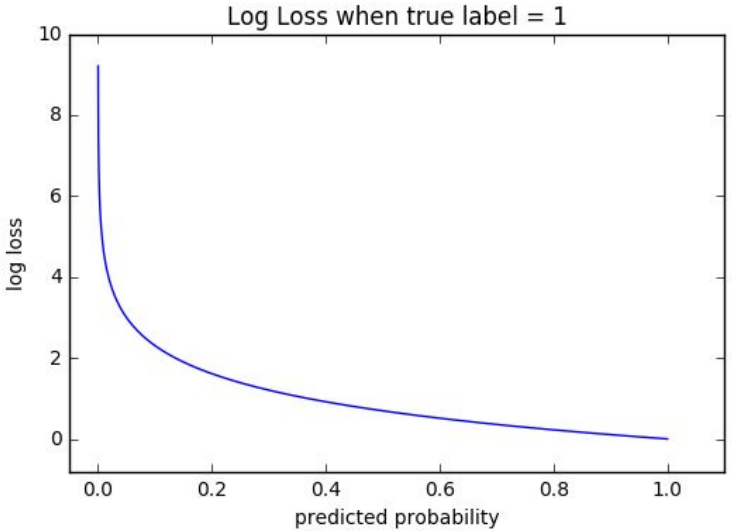


Funciones de activación



Funciones de costo: Cross Entropy

Mide la performance de un modelo de clasificación cuya salida es una probabilidad de 0 a 1. El Cross Entropy Loss aumenta cuando la probabilidad predicha diverge de la etiqueta actual. Un modelo perfecto debería dar una Cross Entropy loss = 0



$$H(P,Q) \stackrel{\text{def}}{=} \sum_{j} -P(j) \log Q(j)$$

$$\mathcal{L}_{CE}(y, \hat{y}) = -\sum_{j=1}^{q} y_j \log \hat{y}_j$$

Funciones de costo: Cross Entropy

Softmax: Multi o binary classification

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

Sigmoid: Binary classification

$$\sigma(\text{x}) = \frac{1}{1 + \exp(-\text{x})}$$

Al final de todas las “layers” de la red neuronal se ubica una función de activación llamada “Softmax” o ‘Sigmoid’ que arroja valores entre 0 y 1 a modo de probabilidad siendo las etiquetas 0 y 1. Los pesos de la red se aprenderán de manera tal que la salida de la softmax o sigmoid coincida lo mejor posible con las etiquetas reales de cada muestra.

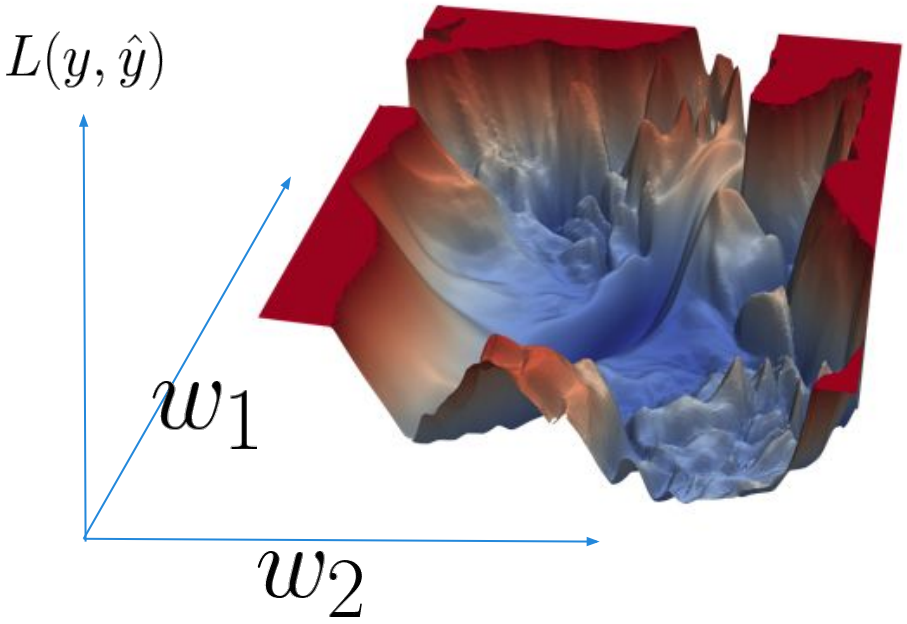
Funciones de costo: Mean Squared Error

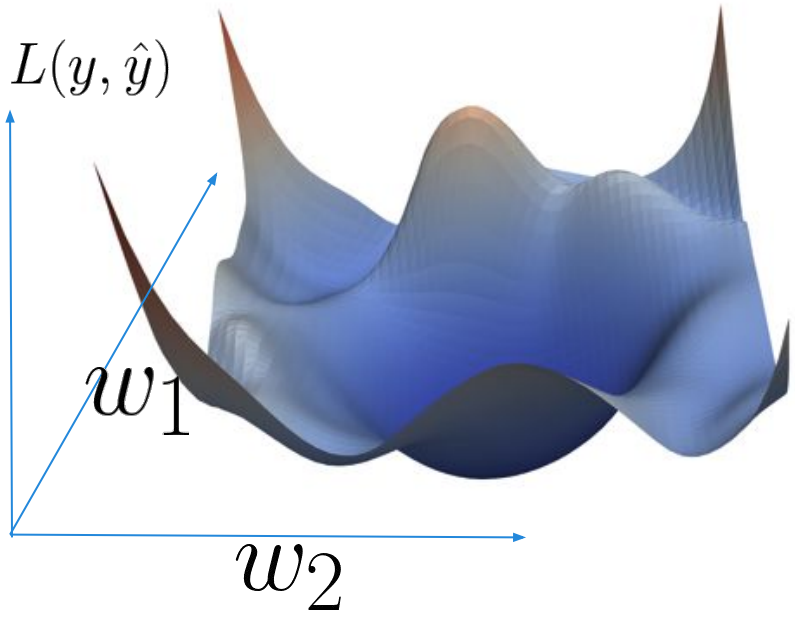
Mide la performance de un modelo de **regresión** cuya salida es la diferencia al cuadrado de los residuos. El MSE además de usarse en problemas de regresión puede usarse en autoencoders donde se busca reconstruir un vector d-dimensional.

$$\text{MSE} = \left\| y_i - y_j \right\|^2$$

Funciones de costo

Las funciones de costo dependen de los parámetros **w** (pesos) del modelo y del dataset en cuestión. Para encontrar los parametros **w** que minimicen el error de prediccion se debe resolver un problema de optimizacion no-lineal. El objetivo es encontrar un vector de parámetros que se encuentre en un minimo local aceptable de la funcion de costo.

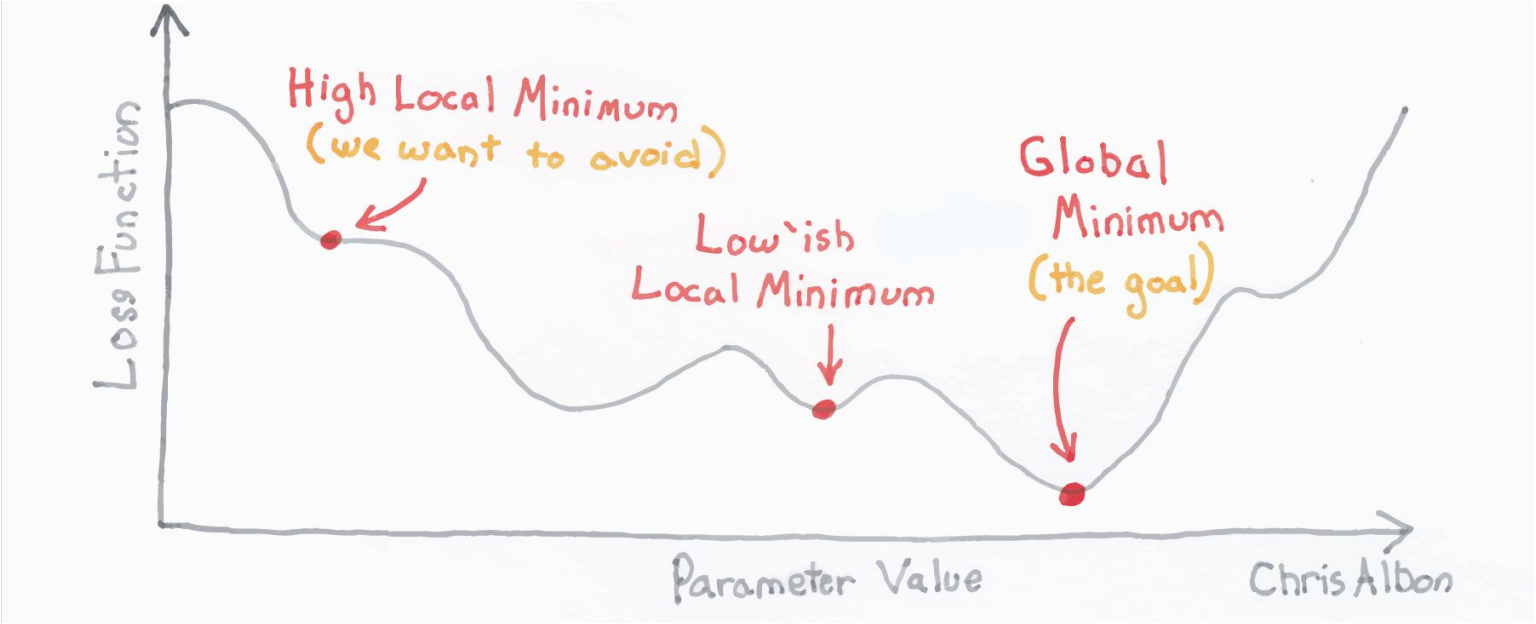




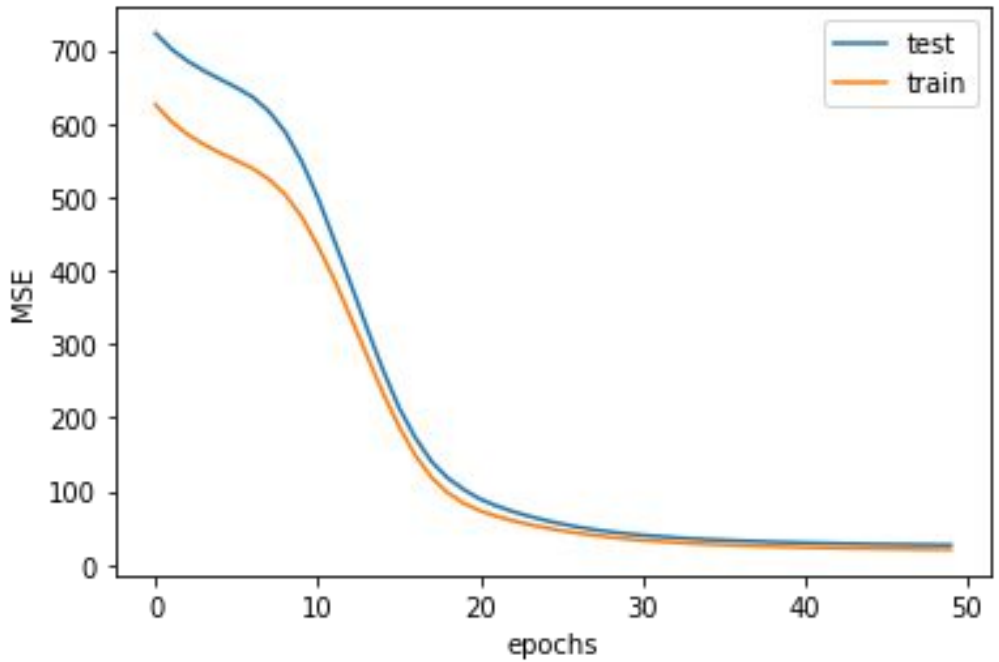
*https://www.jeremyjordan.me

Funciones de costo

Nuestro deseo es encontrar la configuración de pesos **w** que minimiza globalmente la función de costo. A veces no conseguimos ese objetivo ya que la funcion L puede tener multiples minimos locales. aunque podemos llegar a mínimos locales que permiten una performance de clasificación aceptable.



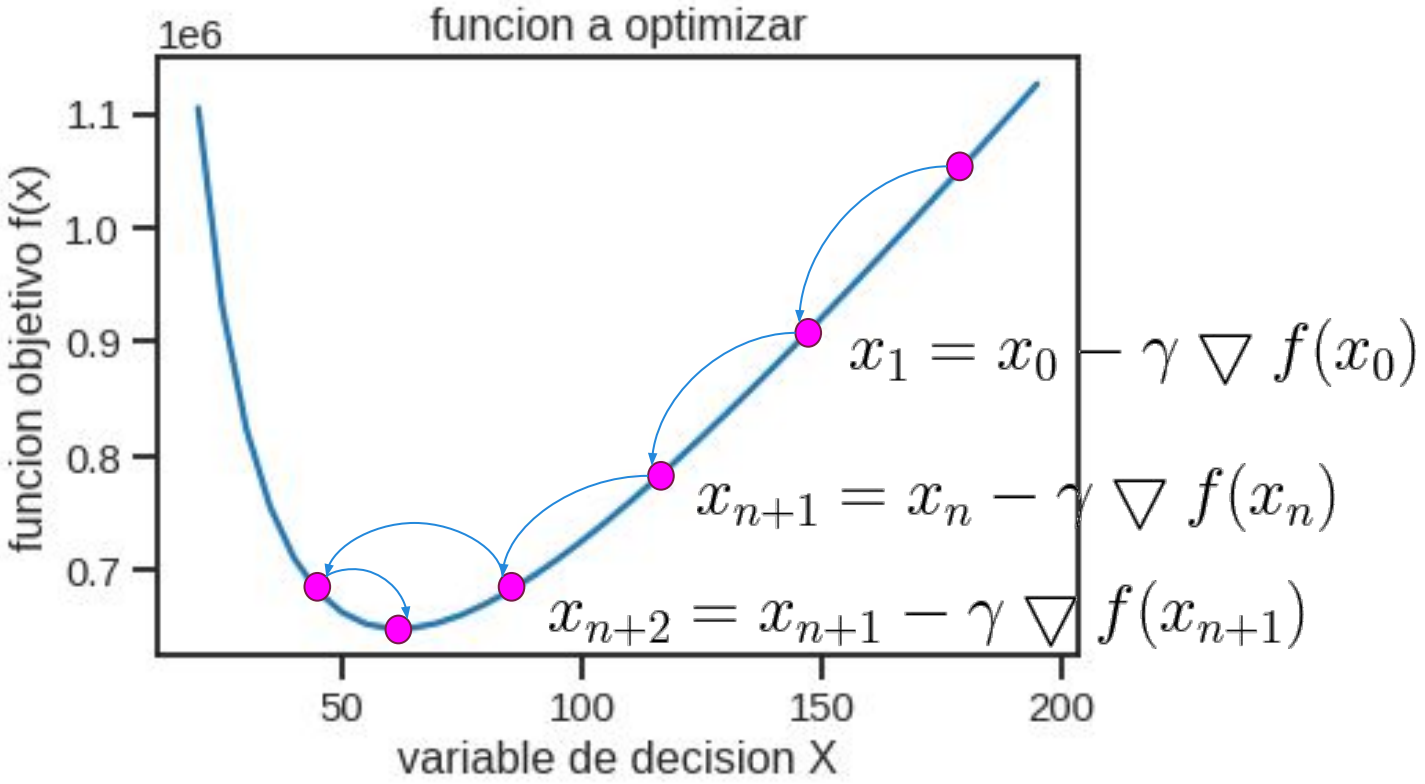
Training Loss



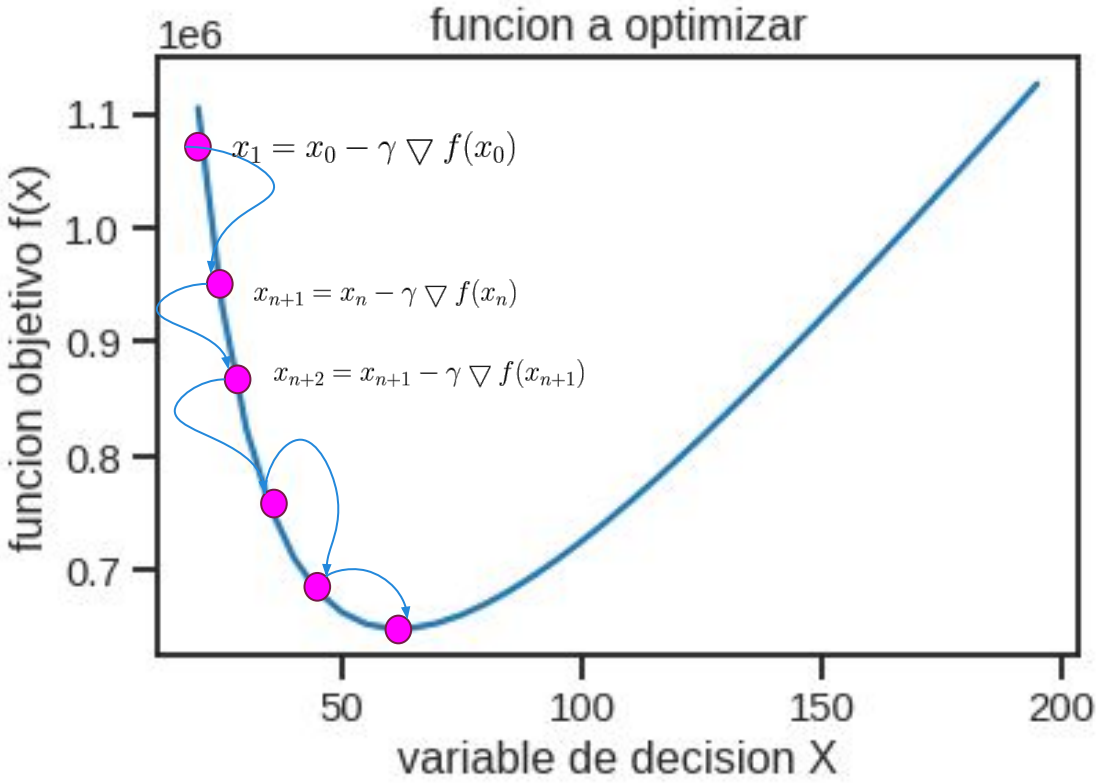
Descenso por gradiente

Metodo para resolver: optimización no lineal

Gradiente descendente



Gradiente descendente



Optimización por gradiente descendiente

$$x \in \mathbb{R}^d \qquad \min_x f(x) \qquad \nabla f(x_0) \qquad \gamma$$

Gradiente de F(x)

Paso de optimización

$$x_1 = x_0 - \gamma \nabla f(x_0)$$

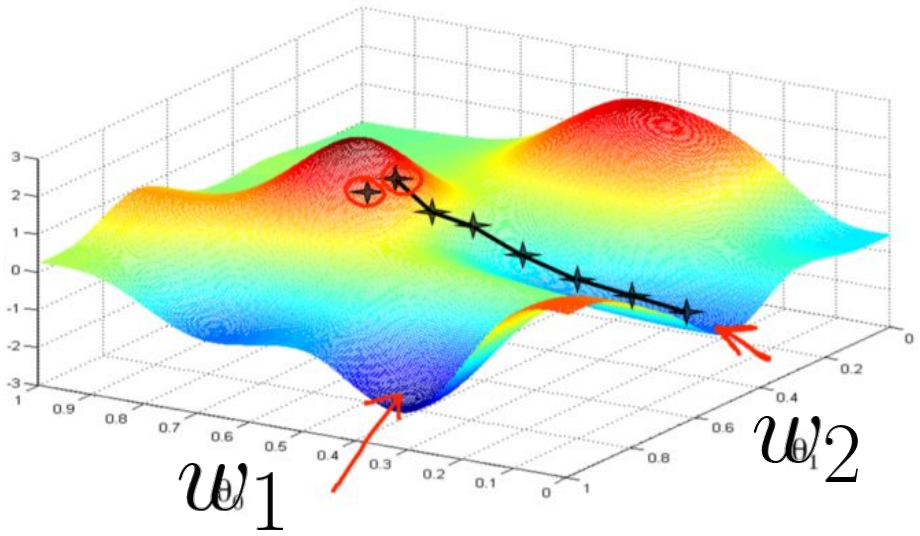
$$x_{n+1} = x_n - \gamma \nabla f(x_n)$$

$$x_{n+2} = x_{n+1} - \gamma \nabla f(x_{n+1})$$

$$f(x_n) > f(x_{n+1})$$

Supongamos que queremos optimizar la función no lineal, continua y diferenciable f(x) definida en el dominio de X. Se partirá desde la solución inicial x0 (llamado semilla) actualizando iterativamente la solución del problema en dirección al gradiente descendiente con un paso *gamma* de manera tal que la nueva solución presente un valor de la función objetivo menor.

Learning Rate y Loss Function



En este caso tenemos una función de costo determinada por dos parámetros (pesos). El Learning Rate se visualiza como los “pasos” que se dan hacia la búsqueda del mínimo. Un Learning rate bajo significa tardar mucho en llegar a un mínimo local. Un LR alto puede significar pasar “por alto” un mínimo y no converger.

Backpropagation & Gradient Descent

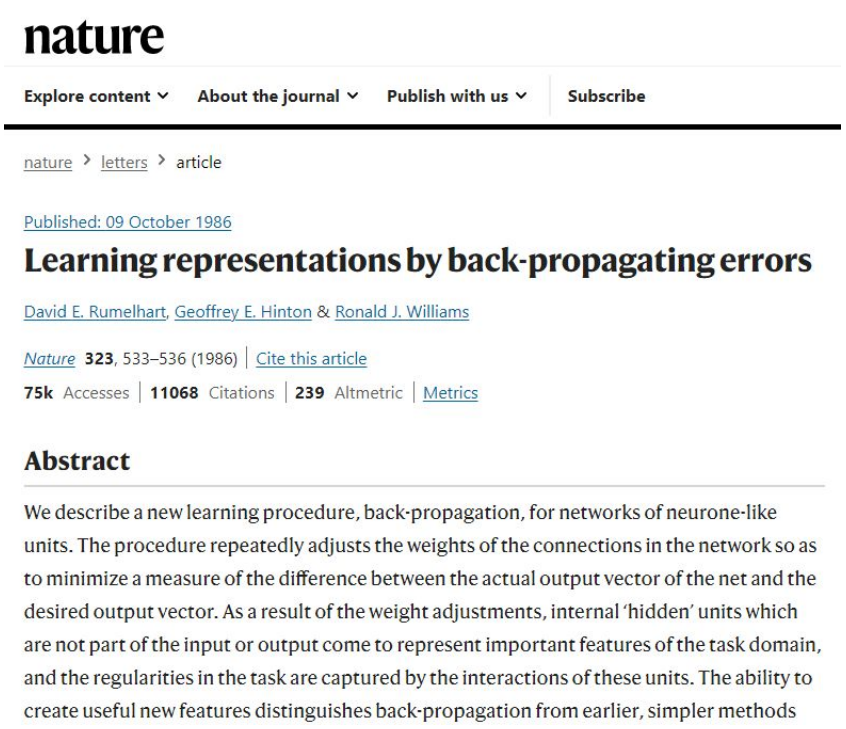
Gradient Descent o Gradiente Descendiente es una técnica de optimización para explorar la función de costo y buscar el mínimo que optimizará nuestra red. El punto de mínimo costo en la función de error implica la combinación de parámetros (pesos) que minimiza el error o la mal clasificación con las muestras de train. Una vez que el gradiente descendiente encuentra un mínimo, detendrá la búsqueda y se quedará en dicha región de la función de costo. La manera en que se busca

$$g = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\Theta} \mathcal{L}\big(z(x_i, \Theta_t), y_i\big)$$

$$\Theta_{t+1} = \Theta_t - \eta g$$

El **Learning Rate** (alpha) es un hiperparámetro que indica cuán rápido se actualizan los pesos en búsqueda de un mínimo.

Backpropagation



nature

Explore content ▾ About the journal ▾ Publish with us ▾ Subscribe

nature › letters › article

Published: 09 October 1986

Learning representations by back-propagating errors

David E. Rumelhart, Geoffrey E. Hinton & Ronald J. Williams

Nature **323**, 533–536 (1986) | Cite this article

75k Accesses | **11068** Citations | **239** Altmetric | Metrics

Abstract

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods

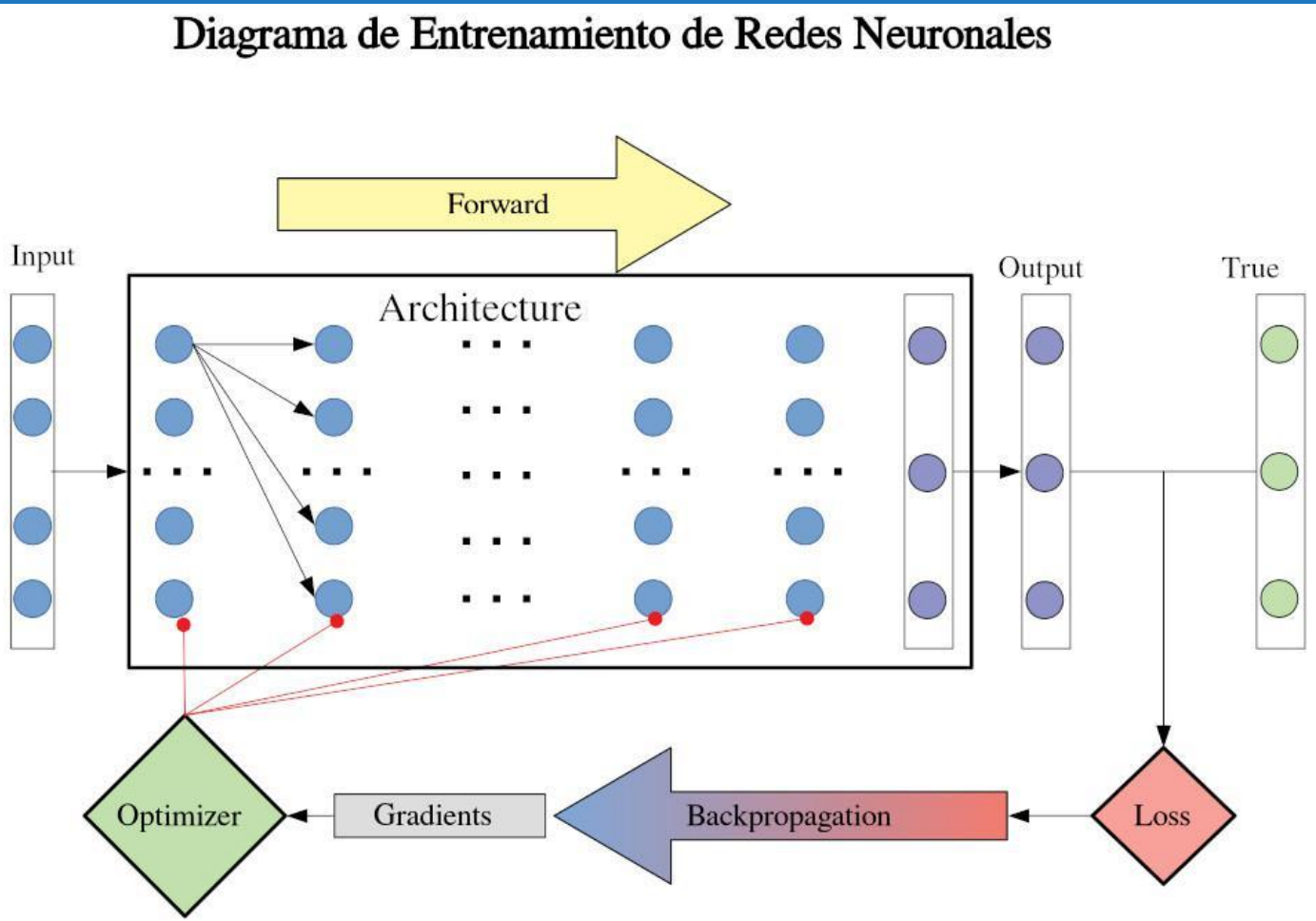
Backpropagation es el **algoritmo** de actualización de pesos de una red neuronal para moverse a favor del gradiente descendiente. Sucede luego que la información fluye desde la entrada input a la salida output (forward propagation) y se conoce si el modelo clasificó bien o mal a cada muestra. Mediante la **regla de la cadena** se calculan los gradientes de la función de costo en función de los parámetros **w** , es decir que se actualizan los pesos que aproximan en dirección al mínimo (local).

Goodfellow, I., Bengio, Y., Courville, A., & Bengio, Y. (2016). *Deep learning* (Vol. 1, p. 2). Cambridge: MIT press. https://www.deeplearningbook.org/contents/mlp.html#pf25

Neural Networks: Components

- Activation functions
- Network architecture
- Loss Function
- Weight optimization technique
- Learning rate

Neural Networks: Diagrama de entrenamiento



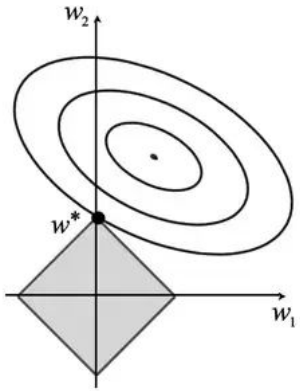
Regularización en NNs

Regularización es penalizar al modelo a la hora de aprender los parámetros internos (los pesos W). No queremos que el modelo aprenda perfectamente los datos de entrenamiento puesto que estos no son 100% representativos de la realidad. La regularización nos permite que el modelo no haga “overfitting”.

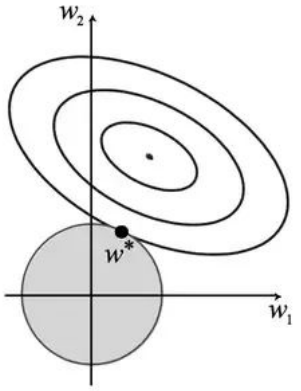
- Norma L1 sobre los pesos
- Norma L2 sobre los pesos
- Dropout en las neuronas
- Batch-Normalization

Regularización en NNs

$$\min_w L(y, \hat{y}) = \min_w L(y, \hat{f}(x))$$



L1



L2

Regularización con norma L2 $\|\mathbf{w}\|_2 = (|w_1|^2 + |w_2|^2)^{1/2}$ $\|\mathbf{w}\|_2 < C$

Regularización con norma L1 $\|\mathbf{w}\|_1 = |w_1| + |w_2|$ $\|\mathbf{w}\|_1 < C$

Regularización en NNs

Norma L1 : Lasso

$$\|\mathbf{w}\|_1 = |w_1| + |w_2| + \cdots + |w_d|$$

Si le imponemos una restricción/penalización a la norma L1 de un vector lo que sucederá es que muchos valores de w tomarán el valor wi = 0 para poder satisfacer la restricción y solo unos pocos serán distinto de cero.

$$\|\mathbf{w}\|_1 < C$$

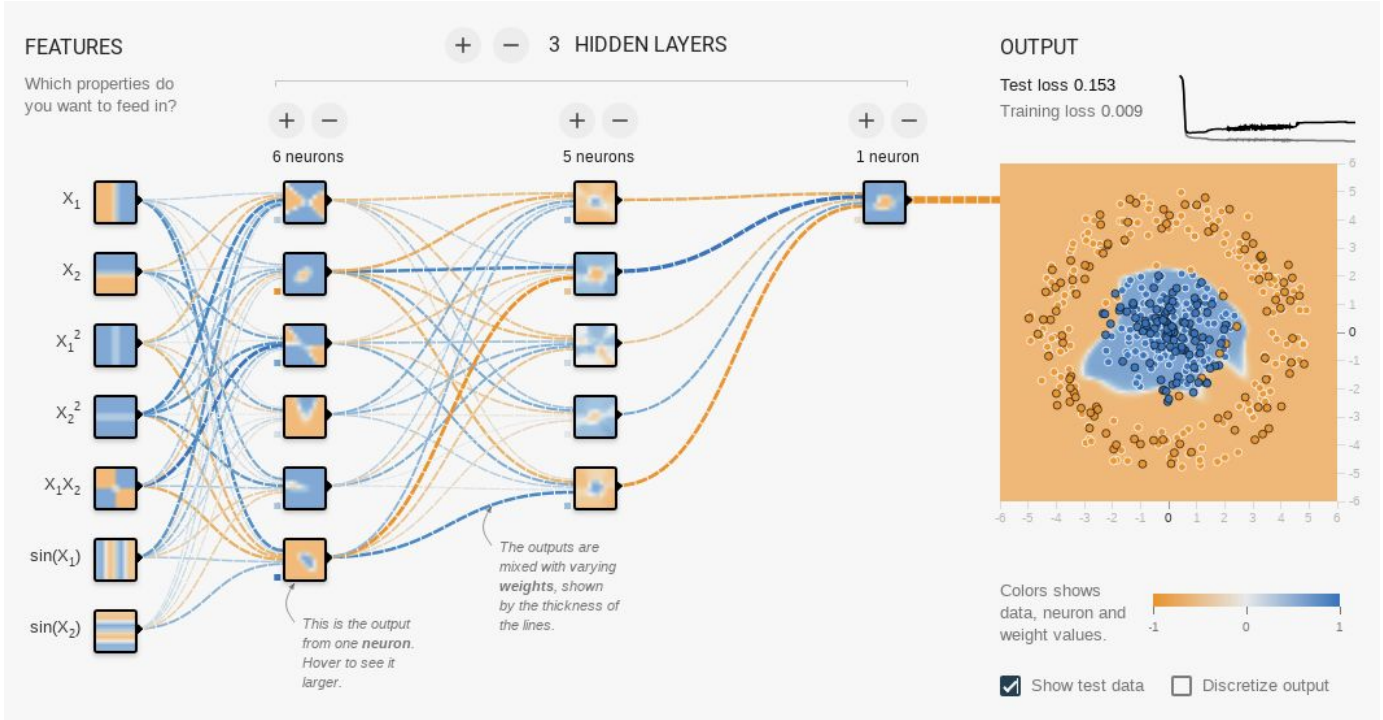
Norma L2 : Ridge

$$\|\mathbf{w}\|_2 = (|w_1|^2 + |w_2|^2 + \cdots + |w_d|^2)^{1/2}$$

Si le imponemos una restricción/penalización a la norma L2 de un vector lo que sucederá es que muchos valores de w tomarán valores cercanos a cero para poder satisfacer la restricción y sólo unos pocos tendrán valores absolutos grandes.

$$\|\mathbf{w}\|_2 < C$$

Neural Networks: Playgroud



*https://playground.tensorflow.org

Deep Learning



Librerías para ANNs





